

Lab 08 — Reentrancy: exploit then patch (EtherBank)

Bài thực hành 08 — Reentrancy: khai thác rồi vá (EtherBank)

Time / Thời lượng: ~50 minutes in class / ~50 phút trên lớp. **Pass condition / Điều kiện đạt:** on the VULNERABLE bank your exploit test drains it to 0; after you PATCH the contract the same attack reverts and every ETH is preserved; you can read the Slither reentrancy-eth finding. / trên bản DỄ TỖN THƯƠNG, test khai thác rút cạn về 0; sau khi VÁ, cùng cuộc tấn công bị revert và giữ nguyên tiền; đọc được cảnh báo reentrancy-eth của Slither.

***Ethics / Đạo đức:** these techniques are for **defense and AUTHORIZED testing only**. Exploit **only** contracts you deploy yourself (local Hardhat chain, or your own TrustKeys test account). Attacking a contract you do not own is a crime. / Kỹ thuật này chỉ dùng để **phòng thủ và kiểm thử được phép**. Chỉ khai thác hợp đồng do **chính bạn** triển khai.*

***Never** paste a real mnemonic or a key holding real funds. Use a **dedicated MetaMask test account** only. / Tuyệt đối không dán mnemonic thật; chỉ dùng tài khoản MetaMask thử riêng.*

Setup / Chuẩn bị:

```
node --version           # must be >= 18 / pha'i >= 18
cd lab/starter           # the TODO project you fill in
npm install              # first run pulls the Hardhat toolbox
npx hardhat test         # the exploit test is a stub – you make it pass
```

The full worked answer is in lab/solutions/ — try the starter before peeking. / lời giải đầy đủ ở lab/solutions/, hãy tự làm trước.

The target / Mục tiêu tấn công

contracts/EtherBank.sol is a toy bank: deposit() credits you, withdraw() pays you back. It holds two seeded bugs. Three honest users (Alice, Bob, Carol) have each deposited 10 ETH → **30 ETH** sits in the bank. You are Eve. / Ngân hàng đồ chơi có hai lỗi. Ba người gửi 10 ETH mỗi người ⇒ 30 ETH trong ngân hàng. Bạn là Eve.

Q0 / Câu 0: Read withdraw(). In what ORDER does it (a) send ETH and (b) update balances? Why is that order the whole problem? / Đọc withdraw(): thứ tự (a) gửi ETH và (b) cập nhật balances là gì? Vì sao thứ tự đó là toàn bộ vấn đề?

Lab 08.1 — Write the Attacker & prove the drain (20') / Viết Attacker & chứng minh rút cạn

Fill the two TODOs in contracts/Attacker.sol: 1. `attack()` — deposit your unit stake, then call `bank.withdraw()` **once**. / gửi stake rồi gọi `withdraw()` một lần. 2. `receive()` — while `address(bank).balance >= unit`, call `bank.withdraw()` **again** (re-enter). / khi ngân hàng còn tiền, gọi lại `withdraw()` (tái nhập).

Then complete `test/Reentrancy.test.js` (Lab 08.1) so it: - deploys your Attacker from Eve with a 1 ETH stake; / triển khai Attacker từ Eve với stake 1 ETH; - calls `attack()` then `collect()`; / gọi `attack()` rồi `collect()`; - asserts **bank balance == 0** (fully drained); / khẳng định số dư ngân hàng = 0; - asserts Eve's **net profit == 30 ETH** (account for gas + her own stake). / lợi nhuận ròng của Eve = 30 ETH (trừ gas & stake).

```
npx hardhat test          # 08.1 must go green – the bank is emptied
```

Q1 / Câu 1: How many times did `withdraw()` execute in a single `attack()` transaction? What stopped the loop? / `withdraw()` chạy bao nhiêu lần trong một giao dịch `attack()`? Điều gì dừng vòng lặp?

Q2 / Câu 2: The vulnerable bank uses `.call{value: bal}("")`. If it had used `transfer()` (2300-gas stipend) instead, would this exact attack still work? Why is "just use transfer" NOT a real fix? / Nếu dùng `transfer()` thay vì `.call`, tấn công này còn chạy? Vì sao "dùng transfer" KHÔNG phải cách vá thật?

Lab 08.2 — Patch it (15') / Vá lỗi

Rewrite `withdraw()` in `contracts/EtherBank.sol` using **Checks-Effects-Interactions**: zero `balances[msg.sender]` **before** the external call. Add a `nonReentrant` guard (inline the `minimal` `one`, `or` `import` `"@openzeppelin/contracts/utils/ReentrancyGuard.sol"`). / Viết lại `withdraw()` theo CEI: đặt số dư = 0 TRƯỚC lời gọi ngoài; thêm guard `nonReentrant`.

Now flip the test assertions (Lab 08.2): the same `attack()` must **revert**, and the bank must still hold **30 ETH**.

```
npx hardhat test          # the drain now FAILS; funds preserved
```

Q3 / Câu 3: With CEI alone (no guard), the re-entered `withdraw()` finds `balances[eve] == 0` and reverts. So why ALSO add the guard? (Hint: cross-function reentrancy — a second function that reads the same state.) / Chỉ CEI đã đủ, vì sao vẫn thêm guard? (Gợi ý: reentrancy chéo hàm.)

Q4 / Câu 4: Your attack reverts with the message "transfer failed", not "ReentrancyGuard: reentrant call". Explain why the guard's revert is "swallowed" by the outer require(ok, ...). / Vì sao thông báo revert là "transfer failed" chứ không phải thông báo của guard?

Lab 08.3 — Run Slither (10') / Chạy Slither

Static analysis finds this class of bug automatically. From lab/solutions/ (or your patched project):

```
pipx install slither-analyzer      # or: pip install --user slither-analyzer
slither .                          # Slither auto-detects Hardhat & compiles
```

Read the **reentrancy-eth** finding on EtherBankVulnerable.withdraw(): "State variables written after the call(s)". Confirm it is **absent** on EtherBankSecure. / Đọc cảnh báo reentrancy-eth; xác nhận nó BIẾN MẤT ở bản đã vá.

***Fallback / Dự phòng:** if Slither won't install on the lab machine, read the saved run in lab/solutions/captured-slither-output.txt. / Nếu không cài được, đọc captured-slither-output.txt.*

Q5 / Câu 5: Slither also reports low-level-calls on the SECURE withdraw(). Is that a vulnerability? Why does the tool still flag it? / Slither cảnh báo low-level-calls ở bản an toàn — đó có phải lỗ hổng không? Vì sao vẫn bị gắn cờ?

Lab 08.4 (bonus) — Access-control bug / Lỗi kiểm soát truy cập

setOwner() has no modifier. Write a property test (test/AccessControl.test.js) that sweeps many signers and shows **any** caller can seize ownership on the vulnerable bank. Then fix it with an onlyOwner modifier (+ zero-check + OwnerChanged event) and prove non-owners are rejected. / Viết property test quét nhiều tài khoản chứng minh ai cũng chiếm được quyền sở hữu; rồi vá bằng onlyOwner.

Q6 / Câu 6: tx.origin vs msg.sender — if onlyOwner used require(tx.origin == owner), how could a malicious contract phish the owner into calling it and steal ownership? / Nếu dùng tx.origin thay msg.sender, hợp đồng độc hại lừa owner thế nào?

Submit / Nộp bài

- npx hardhat test all green on your patched project (exploit proven, then blocked). / test xanh toàn bộ.

- One paragraph: which single line in `withdraw()` was the bug, and the one-line fix. / một đoạn: dòng nào là lỗi và cách sửa.
- (Optional) the Slither reentrancy-eth snippet, before vs after. / (tùy chọn) đoạn cảnh báo Slither trước/sau.

Answer key (Q0–Q6) is in the instructor notes §8. / Đáp án ở ghi chú giảng viên §8.